

tfq.differentiators.ForwardDifference with $n > 1$ parameters

<https://github.com/tensorflow/quantum/issues/710>

ROW 51

tfq.differentiators.ForwardDifference with n > 1 parameters #710

[New issue](#)[Closed](#)

matibilkis opened on Aug 16, 2022

...

Having followed the [documentation example](#), I turned to a more complex circuit, but encounter an error when computing the gradients. Is there any workaround?
Thanks a lot, Matías.

sympy.version = 1.8
tfq.version = 0.7.2
cirq.version = 0.13.1

```
my_op = tfq.get_expectation_op()
linear_differentiator = tfq.differentiators.ForwardDifference(2, 0.01)
op = linear_differentiator.generate_differentiable_op(
    analytic_op=my_op
)
qubit = cirq.GridQubit(0, 0)
circuit = tfq.convert_to_tensor([
    cirq.Circuit([cirq.X(qubit) ** sympy.Symbol('alpha'), cirq.X(qubit) ** sympy.Symbol('beta')])
])

psums = tfq.convert_to_tensor([[cirq.Z(qubit)]])
symbol_values_array = np.array([[0.123, .2]], dtype=np.float32)

# Calculate tfq gradient.
symbol_values_tensor = tf.convert_to_tensor(symbol_values_array)
with tf.GradientTape(persistent=True) as g:
    g.watch(symbol_values_tensor)
    expectations = op(circuit, ['alpha', 'beta'], symbol_values_tensor, psums)

grads = g.gradient(expectations, symbol_values_tensor)
```

and get `ValueError: ('custom_gradient function expected to return', 5, 'gradients but returned', 4, 'instead.')`



Assignees

No one assigned

Labels

No labels

Type

No type

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

No branches or pull requests

Notifications

Customize

[Subscribe](#)

You're not receiving notifications from this thread.

Participants



lockwo on Aug 16, 2022

Contributor

...

Tricky for me to figure out (never interacted with `tf.custom_gradient` before) but simple fix: instead of inputting a string for the symbol names, simply convert them to a tensor. E.g. `['alpha', 'beta']` -> `tf.convert_to_tensor(['alpha', 'beta'])`. If you want to know why, see the end. The following code works for me:

```
my_op = tfq.get_expectation_op()
linear_differentiator = tfq.differentiators.ForwardDifference(2, 0.01)
op = linear_differentiator.generate_differentiable_op(
    analytic_op=my_op
)
qubit = cirq.GridQubit(0, 0)
circuit = tfq.convert_to_tensor([
    cirq.Circuit([cirq.X(qubit) ** sympy.Symbol('alpha'), cirq.X(qubit) ** sympy.Symbol('beta')])
])

psums = tfq.convert_to_tensor([[cirq.Z(qubit)]])
symbol_values_array = np.array([[0.123, .2]], dtype=np.float32)

# Calculate tfq gradient.
exp = tfq.layers.Expectation(differentiator=tfq.differentiators.ForwardDifference(2, 0.01))
symbol_values_tensor = tf.convert_to_tensor(symbol_values_array)
with tf.GradientTape(persistent=True) as g:
    g.watch(symbol_values_tensor)
    expectations = exp(circuit, symbol_names=['alpha', 'beta'], symbol_values=symbol_values_tensor, operators=op)

print(expectations)
grads = g.gradient(expectations, symbol_values_tensor)
print(grads)

symbol_values_tensor = tf.convert_to_tensor(symbol_values_array)
with tf.GradientTape(persistent=True) as g:
    g.watch(symbol_values_tensor)
    expectations = op(circuit, tf.convert_to_tensor(['alpha', 'beta']), symbol_values_tensor, psums)
```

```

    cirq.Circuit([cirq.X(qubit) * sympy.Symbol('alpha'), cirq.X(qubit) * sympy.Symbol('beta')])
])

psums = tfq.convert_to_tensor([[cirq.Z(qubit)]])
symbol_values_array = np.array([[0.123, .2]], dtype=np.float32)

# Calculate tfq gradient.
exp = tfq.layers.Expectation(differentiator=tfq.differentiators.ForwardDifference(2, 0.01))
symbol_values_tensor = tf.convert_to_tensor(symbol_values_array)
with tf.GradientTape(persistent=True) as g:
    g.watch(symbol_values_tensor)
    expectations = exp(circuit, symbol_names=['alpha','beta'], symbol_values=symbol_values_tensor, operators=psums)

print(expectations)
grads = g.gradient(expectations, symbol_values_tensor)
print(grads)

symbol_values_tensor = tf.convert_to_tensor(symbol_values_array)
with tf.GradientTape(persistent=True) as g:
    g.watch(symbol_values_tensor)
    expectations = op(circuit, tf.convert_to_tensor(['alpha','beta']), symbol_values_tensor, psums)

print(expectations)
grads = g.gradient(expectations, symbol_values_tensor)
print(grads)

```

With outputs:

```

tf.Tensor([[0.52784556]], shape=(1, 1), dtype=float32)
tf.Tensor([[[-2.6691551 -2.6691818]], shape=(1, 2), dtype=float32)
tf.Tensor([[0.52784556]], shape=(1, 1), dtype=float32)
tf.Tensor([[[-2.6691551 -2.6691818]], shape=(1, 2), dtype=float32)

```

Why?

The reason why this happens is because the `custom_gradient` function calls `_eager_mode_decorator` which then calls `args = nest.flatten(args)` on the inputs to the function. This will flatten the list into two elements, resulting in 5 elements when it expects 4 [here](#) (hence the error, it originally expected 5 because of the flattening but then later (correctly) only got 4). However, `nest.flatten` does not flatten tensors, keeping the correct number of elements at the beginning therefore aligning it with the 4 it gets later.

Also, just a note, but if you type python after the three ` you can get nice coloring on the code.



matibilkis closed this as **completed** on Aug 17, 2022



matibilkis on Aug 17, 2022

Author ...

Amazing! Thanks a lot Owen. Will definitely add colors to code snippet the next time ;)

